

EXPERIMENT 5

DATA VISUALIZATION USING MATPLOTLIB

Aim

To create and analyze various data visualizations such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots using Matplotlib for effective data interpretation.

Algorithm 1 -Line Chart

Step 1 Import Matplotlib
Step 2 Create dataset
Step 3 Use plot() function
Step 4 Add title and labels
Step 5 Display chart

Program 1:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("C:/Users/91754/Downloads/transactions.csv")

df["Timestamp"] = pd.to_datetime(df["Timestamp"])

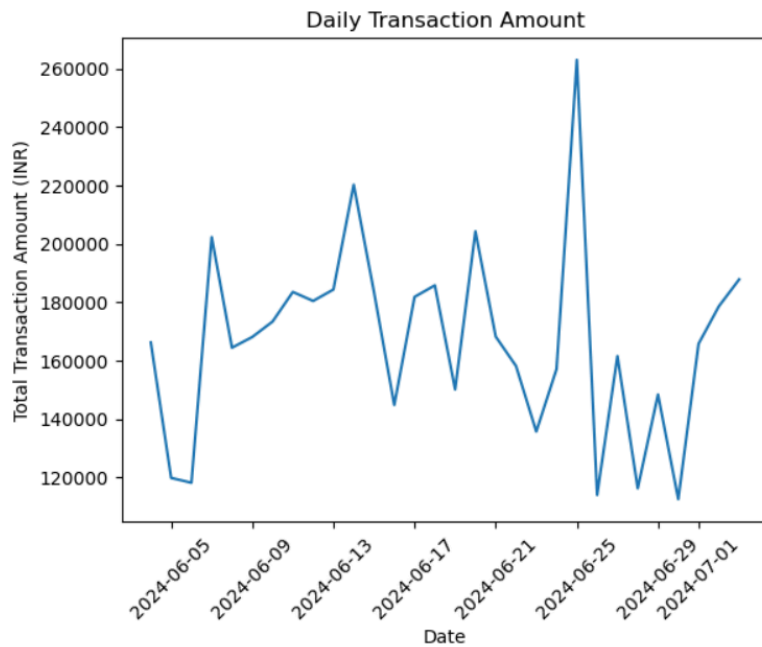
daily_amount = df.groupby(df["Timestamp"].dt.date)["Amount (INR)"].sum()

plt.plot(daily_amount.index, daily_amount.values)

plt.title("Daily Transaction Amount")
plt.xlabel("Date")
plt.ylabel("Total Transaction Amount (INR)")

plt.xticks(rotation=45)
plt.show()
```

Output



Algorithm 2 -Bar Chart

Step 1 Create dataset

Step 2 Use bar() function

Step 3 Add chart labels

Step 4 Display chart

Program 2:

```
status_count = df["Status"].value_counts()
```

```
plt.bar(status_count.index, status_count.values)
```

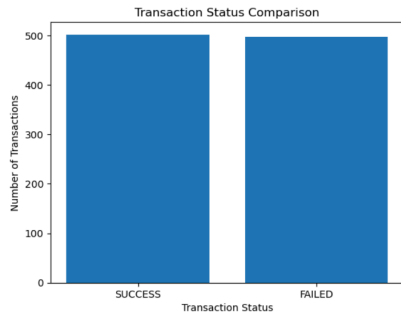
```
plt.title("Transaction Status Comparison")
```

```
plt.xlabel("Transaction Status")
```

```
plt.ylabel("Number of Transactions")
```

```
plt.show()
```

Output:

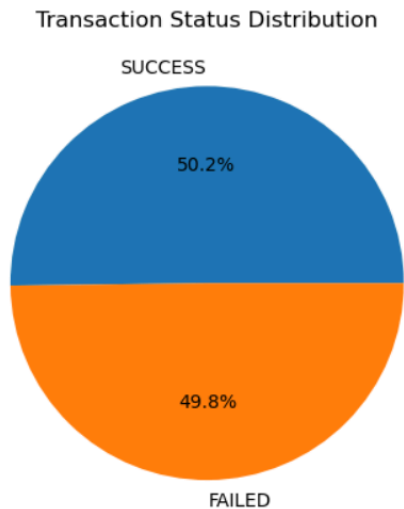


Algorithm 3: Pie Chart

- Step 1: Create the department and employee data.
- Step 2: Use the pie() function to create a pie chart.
- Step 3: Add department names as labels.
- Step 4: Display percentages using the autopct parameter.
- Step 5: Add a title and display the chart.

Program 3:

```
plt.pie(  
    status_count,  
    labels=status_count.index,  
    autopct="%1.1f%%"  
)  
plt.title("Transaction Status Distribution")  
plt.show()
```



Algorithm 4: Histogram

Step 1: Create a numerical dataset containing student marks.

Step 2: Use the `hist()` function to create a histogram.

Step 3: Divide the data into suitable intervals using bins.

Step 4: Add a title and label the X-axis and Y-axis.

Step 5: Display the histogram using `show()`.

Program 4:

```
amount = df["Amount (INR)"]
```

```
plt.hist(amount, bins=5)
```

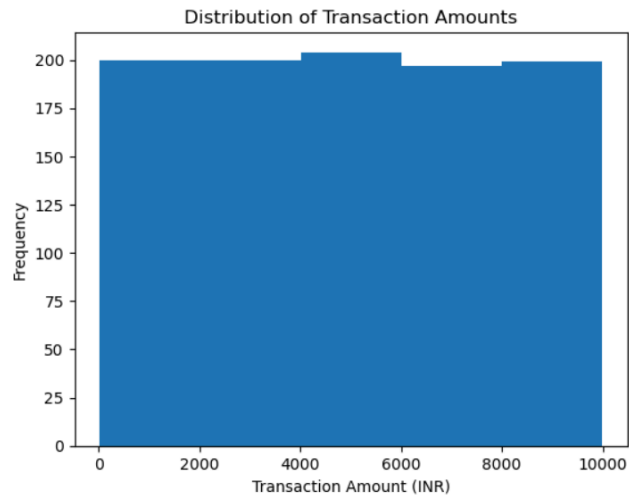
```
plt.title("Distribution of Transaction Amounts")
```

```
plt.xlabel("Transaction Amount (INR)")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

Output:



Algorithm 5: Scatter Plot

Step 1: Create two numerical datasets for attendance and marks.

Step 2: Use the scatter() function to create a scatter plot.

Step 3: Plot attendance on the X-axis and marks on the Y-axis.

Step 4: Add a suitable title and axis labels.

Step 5: Display the scatter plot using show().

Program 5:

```
transaction_number = range(1, len(df) + 1)
```

```
amount = df["Amount (INR)"]
```

```
plt.scatter(transaction_number, amount)
```

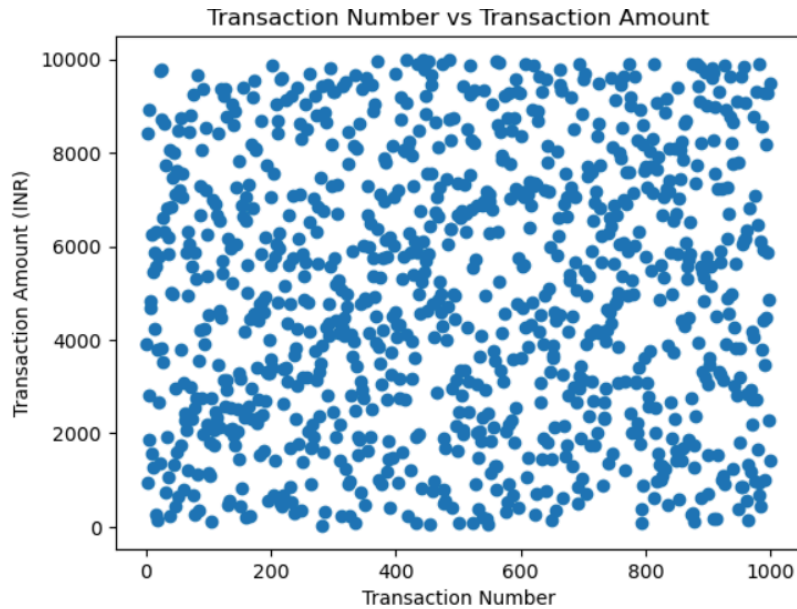
```
plt.title("Transaction Number vs Transaction Amount")
```

```
plt.xlabel("Transaction Number")
```

```
plt.ylabel("Transaction Amount (INR)")
```

```
plt.show()
```

Output:



Algorithm 6: Box Plot

- Step 1: Create a numerical dataset containing salary values.
- Step 2: Use the boxplot() function to create a box plot.
- Step 3: Add a suitable title and label the Y-axis.
- Step 4: Display the box plot using show().
- Step 5: Identify values that appear outside the normal range.

Program 6:

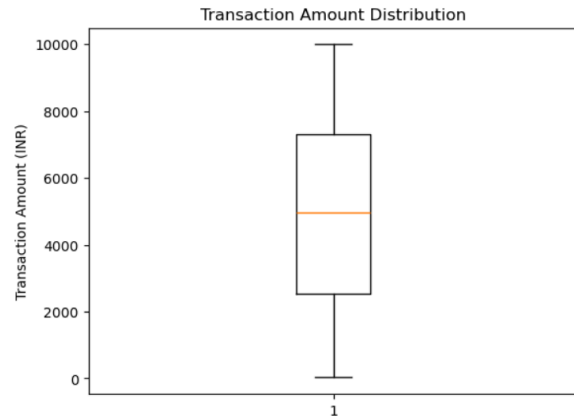
```
plt.boxplot(amount)

plt.title("Transaction Amount Distribution")

plt.ylabel("Transaction Amount (INR)")

plt.show()
```

Output:



Algorithm 7: Multiple Line Charts

Step 1: Create datasets for monthly sales and profit.

Step 2: Use the plot() function to create the sales line.

Step 3: Use the plot() function again to create the profit line.

Step 4: Add labels and a legend to distinguish the two lines.

Step 5: Add a title and display the chart using show().

Program 7:

```
daily_transactions = df.groupby(df["Timestamp"].dt.date).size()
```

```
plt.plot(
```

```
    daily_transactions.index,
```

```
    daily_transactions.values,
```

```
    label="Number of Transactions"
```

```
)
```

```
plt.plot(
```

```
    daily_amount.index,
```

```
    daily_amount.values,
```

```
    label="Total Amount"
```

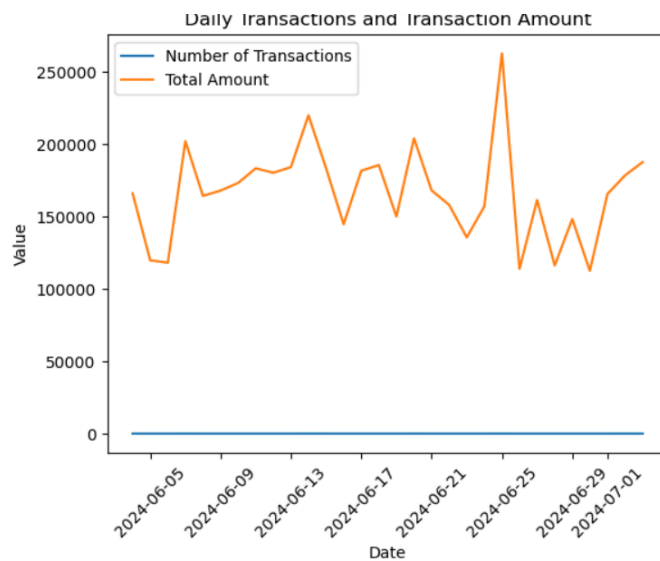
```
)
```

```
plt.title("Daily Transactions and Transaction Amount")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Value")  
  
plt.xticks(rotation=45)  
  
plt.legend()  
  
plt.show()
```

Output:



Result

Thus, various visualization techniques such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots were successfully created using Matplotlib. The graphical representations helped in understanding trends, distributions, comparisons, and relationships within the dataset.